# Lightweight Pothole Detection for Vehicle-Based Road Monitoring

Can a small, fast detector find potholes well enough to report them automatically?

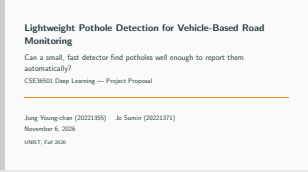
CSE36501 Deep Learning — Project Proposal

Jung Young-chan (20221355)  Jo Sumin (20221371)

November 6, 2026

UNIST, Fall 2026






Hello, we are Jung Young-chan and Jo Sumin. Our project is about lightweight pothole detection for vehicle-based road monitoring. The idea is that future autonomous cars could detect potholes on their own and report them to the road authority right away, so we want to find out whether we can build a detector that is accurate enough while being small and fast enough to run on a car. Today we will cover the problem definition and the datasets.

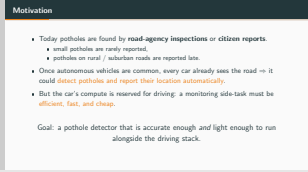
## Motivation

- Today potholes are found by **road-agency inspections** or **citizen reports**.
  - small potholes are rarely reported,
  - potholes on rural / suburban roads are reported late.
- Once autonomous vehicles are common, every car already sees the road ⇒ it could detect potholes and report their location automatically.
- But the car's compute is reserved for driving: a monitoring side-task must be efficient, fast, and cheap.

Goal: a pothole detector that is accurate enough *and* light enough to run alongside the driving stack.








Today, potholes are found in two ways: road agency staff inspect roads, or people report them. But most people do not bother to report small potholes, and on rural or suburban roads reports can come very late. Once autonomous cars are common, every car is effectively a road sensor, so it could detect a pothole and send its location to the road agency immediately. The catch is that the car's computer is busy driving. A pothole detector is only a side task, so it has to be efficient, fast and use little computation. Our question is whether we can build such a model ourselves.

## Problem statement

**Task.** Single-class object detection (bounding boxes) of potholes in vehicle-view road images.

**Constraint.** Low compute: we report accuracy *together with* parameters, GFLOPs and inference speed (FPS).

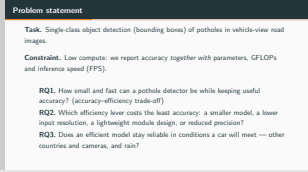
**RQ1.** How small and fast can a pothole detector be while keeping useful accuracy? (accuracy–efficiency trade-off)

**RQ2.** Which efficiency lever costs the least accuracy: a smaller model, a lower input resolution, a lightweight module design, or reduced precision?

**RQ3.** Does an efficient model stay reliable in conditions a car will meet — other countries and cameras, and rain?








Formally, the task is single-class pothole detection with bounding boxes, under a compute constraint. So every result will be reported as accuracy together with model size, GFLOPs and speed. This gives three research questions. First: how small and fast can the detector get before accuracy becomes useless? Second: there are several ways to make a model cheaper, a smaller network, smaller input images, a lighter module design, or lower numerical precision. Which one hurts accuracy least? And third: a car will drive in other countries and in the rain, so does the efficient model still work there?

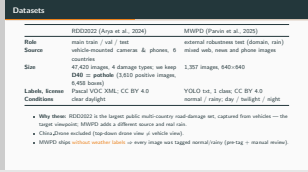
# Datasets

| | RDD2022 (Arya et al., 2024) | MWPD (Parvin et al., 2025) |
|---|---|---|
| **Role** | main train / val / test | external robustness test (domain, rain) |
| **Source** | vehicle-mounted cameras & phones, 6 countries | mixed web, news and phone images |
| **Size** | 47,420 images, 4 damage types; we keep **D40 = pothole** (3,610 positive images, 6,458 boxes) | 1,357 images, 640×640 |
| **Labels, license** | Pascal VOC XML; CC BY 4.0 | YOLO txt, 1 class; CC BY 4.0 |
| **Conditions** | clear daylight | normal / rainy; day / twilight / night |

- **Why these:** RDD2022 is the largest public multi-country road-damage set, captured from vehicles — the target viewpoint; MWPD adds a different source and real rain.
- China_Drone excluded (top-down drone view $\neq$ vehicle view).
- MWPD ships without weather labels $\Rightarrow$ every image was tagged normal/rainy (pre-tag + manual review).








We use two datasets. RDD2022 is the large multi-country road damage dataset from the CRDDC challenge, about forty-seven thousand images captured from vehicles, which is exactly the viewpoint we care about. It has several damage classes; we keep only D40, the pothole class, and we drop the China drone subset because it is shot from above. MWPD, the Multi-Weather Pothole Detection dataset, is used only for testing: it comes from a different source and includes real rain, so it tells us how the model transfers. MWPD has no weather labels in its files, so we tagged every image ourselves.

# What the data looks like



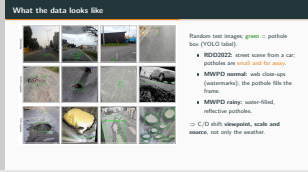

Random test images; green = pothole box (YOLO label).

- **RDD2022:** street scene from a car; potholes are small and far away.
- **MWPD normal:** web close-ups (watermarks); the pothole fills the frame.
- **MWPD rainy:** water-filled, reflective potholes.

⇒ C/D shift **viewpoint, scale and source**, not only the weather.

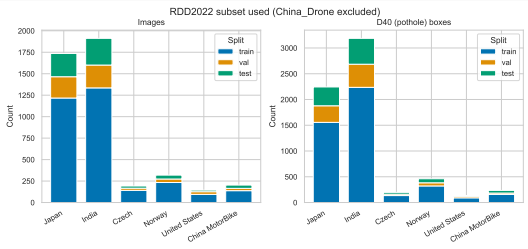




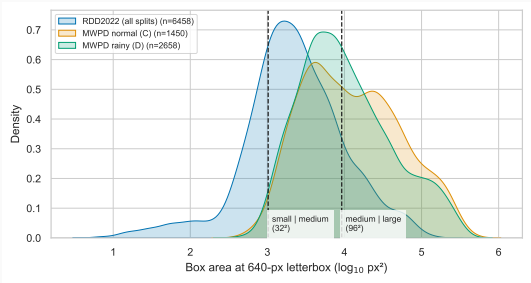


This is what the data actually looks like. The top row is RDD2022: the camera sits in a car and sees the whole street, so the pothole is usually a small box near the bottom of the image. The middle row is MWPD in normal weather: mostly close-up photos collected from the web, some with stock-photo watermarks, where the pothole fills a large part of the image. The bottom row is MWPD in the rain: potholes are filled with water and reflect the sky, so they look quite different. This is why the MWPD test sets measure more than rain: the viewpoint, the object size and the image source all change as well.

# Dataset analysis




Only 10% of annotated RDD images contain a pothole; Japan + India hold 81% of them.



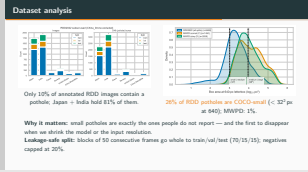

26% of RDD potholes are COCO-small ($< 32^2$ px at 640); MWPD: 1%.

**Why it matters:** small potholes are exactly the ones people do not report — and the first to disappear when we shrink the model or the input resolution.

**Leakage-safe split:** blocks of 50 consecutive frames go whole to train/val/test (70/15/15); negatives capped at 20%.



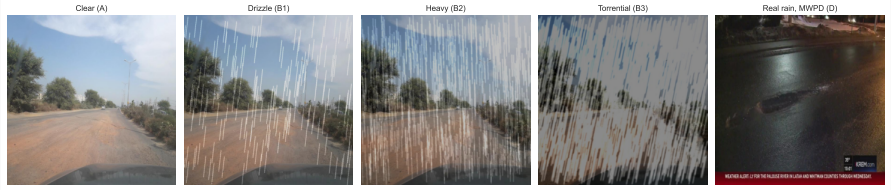




Two observations. On the left: potholes are rare in RDD; only about one in ten annotated images contains one, and most come from Japan and India. On the right, and most important for this project: about a quarter of the RDD potholes are small objects, less than thirty-two pixels across at the usual input size. Those small potholes are exactly the ones people never report, and they are also the first thing a detector loses when we make it cheaper, for example by lowering the input resolution. So we will always report accuracy for small objects separately. Finally, RDD contains consecutive frames, so we split by blocks of fifty frames to keep near-identical images out of both train and test.

# Evaluation design

| ID | Composition | Images | Purpose |
|---|---|---|---|
| A | RDD2022 own test split (clear) | 708 | **main accuracy** |
| B1/B2/B3 | *same images* as A + synthetic drizzle/heavy/torrential | 708 each | rain robustness |
| C | MWPD normal weather | 681 | new source / camera |
| D | MWPD rainy | 676 | real rain |

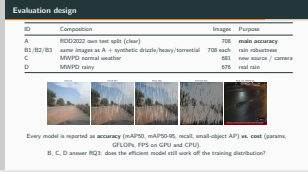



Every model is reported as **accuracy** (mAP50, mAP50-95, recall, small-object AP) **vs. cost** (params, GFLOPs, FPS on GPU and CPU).

B, C, D answer RQ3: does the efficient model still work off the training distribution?







For evaluation, set A, our clean RDD test split, measures the main accuracy. Sets B1 to B3 are the very same images with synthetic rain of increasing strength, and C and D are MWPD images in normal weather and in real rain. Those three groups are robustness checks for the third research question. Every model is reported on two axes: accuracy, including small-object accuracy, and cost, meaning parameters, GFLOPs and frames per second on both a GPU and a CPU.

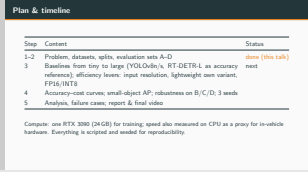
# Plan & timeline

| Step | Content | Status |
|---|---|---|
| 1–2 | Problem, datasets, splits, evaluation sets A–D | done (this talk) |
| 3 | Baselines from tiny to large (YOLOv8n/s, RT-DETR-L as accuracy reference); efficiency levers: input resolution, lightweight own variant, FP16/INT8 | next |
| 4 | Accuracy–cost curves; small-object AP; robustness on B/C/D; 3 seeds | |
| 5 | Analysis, failure cases; report & final video | |

Compute: one RTX 3090 (24 GB) for training; speed also measured on CPU as a proxy for in-vehicle hardware. Everything is scripted and seeded for reproducibility.








Steps one and two, the problem and the data, are finished and are what we presented today. Next we will train baselines from very small to large, and then apply efficiency levers: lower input resolution, a lightweight variant of our own, and reduced precision. The main result will be accuracy-versus-cost curves, with small-object accuracy and the robustness sets. Since the target is a car rather than a big GPU, we will also measure speed on a CPU. Thank you.